

STATE VECTOR SIMULATION OF DISTRIBUTED KQD

AADITHYA IYER

AADITHYAIYER@IISC.AC.IN

MOTIVATION

- Time independent Schrödinger equation for many-body systems
 - Uses in condensed matter physics, quantum chemistry & high energy physics
 - Time independent Schrödinger equation: $\hat{H}\psi(x) = E\psi(x)$
 - $\hat{H}$: Hamiltonian operator of size $2^N \times 2^N$ (N is the number of atoms being observed)
 - $\psi(x)$: Eigenfunction of size 2^N
 - **Objective**: To find the Ground State Energy($\lambda_{\min}$) of the Hamiltonian $\hat{H}$
- Classical Krylov Subspace Method:
 - Construct a Krylov Subspace as: $\mathcal{K}^r = \text{span} \{ |v\rangle, A|v\rangle, A^2|v\rangle, \dots, A^{r-1}|v\rangle \}$
 - Project Hamiltonian into the Krylov Subspace and solve the generalized EVP $\tilde{H}c = E\tilde{S}c$
 - **Problem**: Hamiltonian representing the state of just 20 atoms, is a matrix of size $2^{20} \times 2^{20}$!!
 - The size of the Hamiltonian matrix grows exponentially as molecule size increases!
- **Theorem**: The Krylov Subspace generated by the Hamiltonian H acting on state $|\psi_0\rangle$ is **equivalent** to the subspace generated by $U(kt)|\psi_0\rangle = e^{-iH(kt)}|\psi_0\rangle$!

INTRO TO KRYLOV QUANTUM DIAGONALIZATION

$$U = e^{-iH dt}$$

$$U^k = e^{-iH(k dt)}$$

$$\mathcal{K}_U^r = \text{span} \{ |\psi\rangle, U|\psi\rangle, U^2|\psi\rangle \dots U^{r-1}|\psi\rangle \}$$

$$\tilde{H}_{mn} = \langle \psi_m | H | \psi_n \rangle$$

$$= \langle \psi e^{iHt_m} | H | \psi e^{-iHt_n} \rangle$$

$$= \langle \psi e^{iHm dt} | H | \psi e^{-iHn dt} \rangle$$

$$\tilde{S}_{mn} = \langle \psi_m | \psi_n \rangle$$

Trotterization

Expectation Values
(First Row!)

Outer loop –
for k = 0 to krylov_dim

k=0

analytical
 $S(0)=1$

$$H(0) = \langle \psi_0 | H | \psi_0 \rangle$$

k=1,2,3

Efficient
Hadamard Test
get Re(S), Im(S)
get Re(H), Im(H)

Toeplitz assembly

$S[i][j] = s_row[|i-j|]$
 $H[i][j] = H_row[|i-j|]$
conj for $i > j$

solve $H \cdot c = E \cdot S \cdot c$

generalised eig
lowest eigenvalue
= E_0 estimate

Finally the Generalized Eigenvalue Problem is solved classically as: $\tilde{H} \vec{c} = E \tilde{S} \vec{c}$

INTRODUCTION TO STATE VECTOR SIMULATION

- The Quantum State of an N qubit quantum system:
$$|\psi\rangle = \alpha_0|00\dots 0\rangle + \alpha_1|00\dots 1\rangle + \dots + \alpha_{2^N-1}|11\dots 1\rangle$$
 - **Ex:** N atoms, each either excited ($|1\rangle$) or in its ground ($|0\rangle$) state **2^N possible configs**
 - Here, $\alpha_0, \alpha_1, \dots, \alpha_{2^N-1}$ are called amplitudes
- **Amplitudes are Complex Numbers!**
 - **Magnitude:** The square root of probability of observing state i, where $P_i = \text{Re}(\alpha_i)^2 + \text{Im}(\alpha_i)^2$
 - **Phase:** Argument/Direction of Complex Amplitude;
 - Doesn't affect the probability of observing that single state P_i
 - Determines how α_i interferes with other amplitudes α_j when gates mix them.
 - **(How are Gates Applied?)**
- Stored in an array of size 2^N , which is called a State Vector
- Simulating quantum circuits on classical computers through a series of transformations to evolve our state vector is known as state vector simulation.

INTRODUCTION TO CUDAQ

- **Quantum Simulator Developed by Nvidia**

- Efficient implementation of quantum circuits
- C++ Backend
- **Utilities:**
 - Applying quantum gates to state vector
 - Calculating Expectation value
- Good baseline code for performing KQD

- **In this code:**

- `cudaq::observe` function:
 - Calculates expectation value $\langle \psi_0 | PU | \psi_0 \rangle$

- **Miscellaneous Concepts:**

- Heisenberg Chain $H = \sum J_{ij}(X_i X_j + Y_i Y_j + Z_i Z_j)$
- Inputs: J_{ij} , n_q
- Nearest Neighbour Interaction: $J_{ij} = 0$ if not NN!
- Outputs: Ground State Energy $\lambda_{\min}$

```
// Minimal CUDA-Q Example: Hadamard + Expectation Value
#include <cudaq.h>

// 1. Define a kernel
struct HadamardCircuit {
    void operator()(int n_qubits) __qpu__ {

        // Allocate n_qubits qubits – all initialised to |0>
        cudaq::qvector q(n_qubits);
        // Apply Hadamard to qubit 0 only.
        // Statevector after this:
        // [1/√2, 0, 0, 0, 0, 0, 0, 0, 1/√2, 0, 0, 0, 0, 0, 0, 0]
        // ^--- |0000>                      ^--- |1000>
        // All other qubits remain in |0>.
        cudaq::h(q[0]);
    }
};

// 2. Call cudaq::observe() to obtain expectation values.
int main() {

    const int n_qubits = 4;    // change freely
    // (Z) = P(qubit 0 = |0>) - P(qubit 0 = |1>)
    //       = 1/2 - 1/2
    //       = 0.0
    auto Z0 = cudaq::spin::z(0);

    // cudaq::observe() internally simulates the circuit
    // computes <ψ|Z|ψ> exactly
    // Internally: runs reduce P(0)-P(1) over statevector.
    auto result = cudaq::observe(HadamardCircuit{}, Z0, n_qubits);

    // .expectation() extracts the scalar – returns 0.0
    printf("<Z0> is (expected: 0.0):=", result.expectation());

    return 0;
}
```

PROBLEM STATEMENT

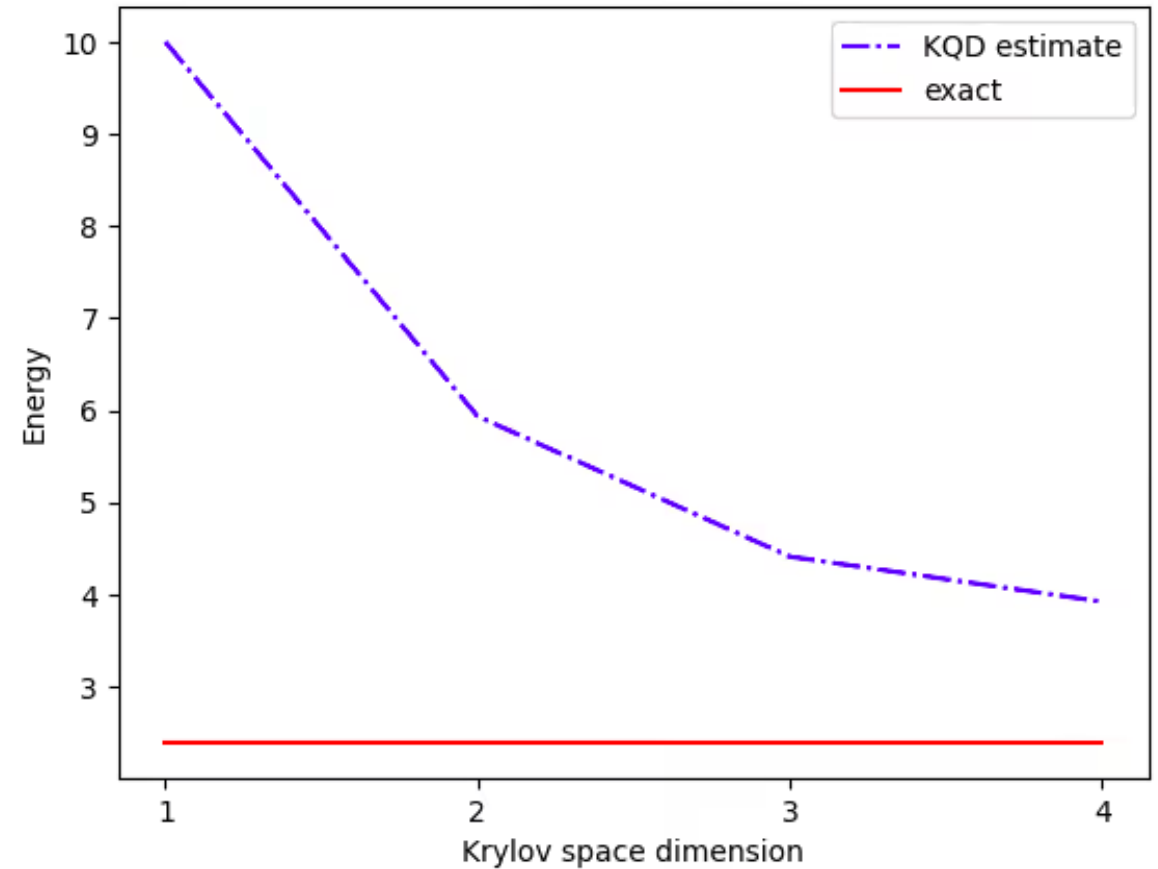
- **Main research gaps:**

- Current algorithm uses **only 1 QPU(IBM Fez)**
 - Takes around 17 minutes
- **Parallelize Independent Computations**
 - Parallel Construction Krylov Subspace
 - Each Krylov Vector Uses $|\psi_0\rangle$ **not the previously generated krylov vector!**
- **Research on State Vector Simulation**

- **Deliverables:**

- CUDAQ Implementation of KQD
- Utilise multiple QPUs together! (**e.g: ibm_kingston, fez, marrakesh**)
 - Emulation of algorithm on GPUs(CUDA)
 - Emulation of algorithm on CPUs(OpenMP)
- Optimisations to current algorithm:
 - Make copies of the evolved state vector instead of 4 separate runs

Estimating Ground state energy with Krylov Quantum Diagonalization



METHODOLOGY: DISTRIBUTED KQD

(Define State Vector of size 2^N)

- **State Preparation(PREP) Steps 1 & 2:**

- Hadamard(q_0)
- CX(q_0 as control as q_{mid} as target)

- **U(t) (Step 3)**

- Trotterisation (dt) = $(\pi / ||H||_2) / n_{steps}$
- (Series of 2-qubit gates)
- RXX $\rightarrow$ RYY $\rightarrow$ RZZ
- RZZ $\rightarrow$ RYY $\rightarrow$ RXX
- Operation pair done $n_{steps}/2$ times

- **UNPREP(Pauli) (Step 4)**

- CX(q_0 as control and q_{mid} as target)
- Basis change operation
 - ($H(q_0)$ or $H(q_0) + S^+(q_0)$)

- **Finally, Calculate Expectation Value!**

Questions:

How to Apply Gates to our State Vector?

How to calculate the Expectation Value

$$\text{Step 0: } |\Psi\rangle = |0\rangle|0\rangle^N$$

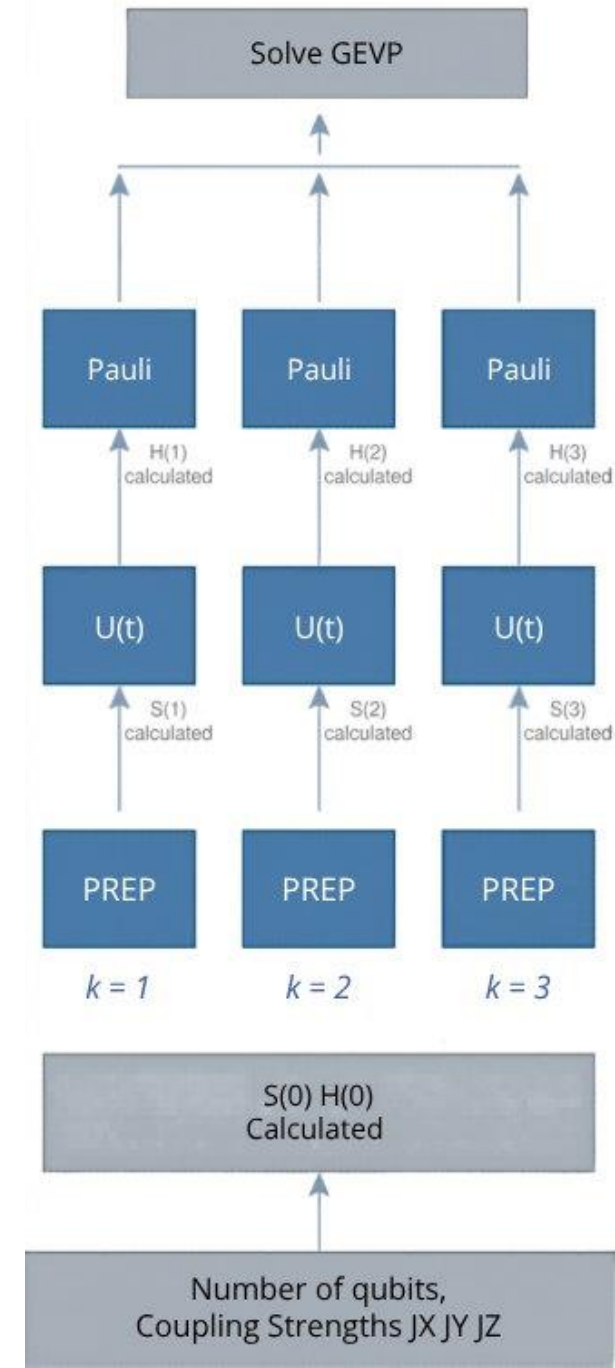
$$\text{Step 1: } |\Psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|0\rangle^N$$

$$\text{Step 2: } |\Psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle|0\rangle^N + |1\rangle|\psi_0\rangle)$$

$$\text{Step 3: } |\Psi\rangle = \frac{1}{\sqrt{2}}(e^{i\phi}|0\rangle|0\rangle^N + |1\rangle U|\psi_0\rangle)$$

$$\begin{aligned} \text{Step 4: } |\Psi\rangle &= \frac{1}{\sqrt{2}}(e^{i\phi}|0\rangle|\psi_0\rangle + |1\rangle U|\psi_0\rangle) \\ &= \frac{1}{2}(|+\rangle(e^{i\phi}|\psi_0\rangle + U|\psi_0\rangle) + |-\rangle(e^{i\phi}|\psi_0\rangle - U|\psi_0\rangle)) \\ &= \frac{1}{2}(|+\rangle + i)(e^{i\phi}|\psi_0\rangle - iU|\psi_0\rangle) + |-\rangle(e^{i\phi}|\psi_0\rangle + iU|\psi_0\rangle) \\ \langle X \otimes P \rangle &= \frac{1}{4} \left[(e^{-i\phi}\langle\psi_0| + \langle\psi_0|U^\dagger)P(e^{i\phi}|\psi_0\rangle + U|\psi_0\rangle) \right. \\ &\quad \left. - (e^{-i\phi}\langle\psi_0| - \langle\psi_0|U^\dagger)P(e^{i\phi}|\psi_0\rangle - U|\psi_0\rangle) \right] \\ &= \text{Re}[e^{-i\phi}\langle\psi_0|PU|\psi_0\rangle] \end{aligned}$$

$$\begin{aligned} \langle Y \otimes P \rangle &= \frac{1}{4} \left[(e^{-i\phi}\langle\psi_0| + i\langle\psi_0|U^\dagger)P(e^{i\phi}|\psi_0\rangle - iU|\psi_0\rangle) \right. \\ &\quad \left. - (e^{-i\phi}\langle\psi_0| - i\langle\psi_0|U^\dagger)P(e^{i\phi}|\psi_0\rangle + iU|\psi_0\rangle) \right] \\ &= \text{Im}[e^{-i\phi}\langle\psi_0|PU|\psi_0\rangle]. \end{aligned}$$



SINGLE QUBIT GATE

Our State vector is:

$$|\psi\rangle = \alpha_0|00\dots 0\rangle + \alpha_1|00\dots 1\rangle + \dots + \alpha_{2^N-1}|11\dots 1\rangle$$

- Our first task is to group the elements in our state vector into 2^{n-1} pairs.
- These are small 2x1 vectors, and we apply our matrix Q to every single one of these 2x1 vectors.

Definitions

i_0, i_1 statevector indices coupled by the gate; they differ *only* at bit position q

$$Q = \begin{pmatrix} q_{11} & q_{12} \\ q_{21} & q_{22} \end{pmatrix} \quad \text{unitary } 2 \times 2 \text{ gate on qubit } q, \quad \text{e.g. } H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Index mapping ($\text{iter} \in \{0, \dots, 2^{n-1}-1\}$):

$$lo = \text{iter} \bmod 2^q, \quad hi = \left\lfloor \frac{\text{iter}}{2^q} \right\rfloor$$

$$i_0 = hi \cdot 2^{q+1} + lo, \quad i_1 = i_0 + 2^q$$

Splitting the iterator around bit q inserts 0 or 1 at that position, selecting exactly the two amplitudes that differ only at qubit q .

GPU

CPU

GPU — CUDA Kernel

```
tid = (blockIdx × 256 + threadIdx) × C
```

```
// one thread per amplitude pair
```

```
compute lo, hi from tid
```

```
compute i0, i1
```

$$\alpha'_{i_0} \leftarrow q_{11} \cdot \alpha_{i_0} + q_{12} \cdot \alpha_{i_1}$$

$$\alpha'_{i_1} \leftarrow q_{21} \cdot \alpha_{i_0} + q_{22} \cdot \alpha_{i_1}$$

CPU — OpenMP

```
#pragma omp parallel for
```

```
// one iteration per amplitude pair
```

```
for iter = 0 to 2n-1 - 1:
```

```
    compute lo, hi from iter
```

```
    compute i0, i1
```

$$\alpha'_{i_0} \leftarrow q_{11} \cdot \alpha_{i_0} + q_{12} \cdot \alpha_{i_1}$$

$$\alpha'_{i_1} \leftarrow q_{21} \cdot \alpha_{i_0} + q_{22} \cdot \alpha_{i_1}$$

2-QUBIT GATE

2-Qubit Gate Implementation:

- Instead of pairs, we obtain groups of 4 for each thread to operate on.
- Example Gates:

$$R_{YY}(\theta) = \exp\left(-i\frac{\theta}{2}Y \otimes Y\right) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & 0 & 0 & i\sin\left(\frac{\theta}{2}\right) \\ 0 & \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) & 0 \\ 0 & -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) & 0 \\ i\sin\left(\frac{\theta}{2}\right) & 0 & 0 & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}$$

$$R_{XX}(\theta) = \exp\left(-i\frac{\theta}{2}X \otimes X\right) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & 0 & 0 & -i\sin\left(\frac{\theta}{2}\right) \\ 0 & \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) & 0 \\ 0 & -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) & 0 \\ -i\sin\left(\frac{\theta}{2}\right) & 0 & 0 & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}$$

$$CX_{q_0, q_1} = I \otimes |0\rangle\langle 0| + X \otimes |1\rangle\langle 1| = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

Ref: <https://arxiv.org/abs/1601.07195> qHiPSTER: The Quantum High Performance Software Testing Environment
<https://quantum.cloud.ibm.com/docs/en/api/qiskit/>

Definitions

$i_{00}, i_{01}, i_{10}, i_{11}$ four statevector indices coupled by the gate; they span all $\{0, 1\}$ combinations at bit positions lo and hi

$lo = \min(q_1, q_2), \quad hi = \max(q_1, q_2), \quad c = \cos(\theta/2), \quad s = \sin(\theta/2)$ (e.g. R_{XX}, R_{YY}, R_{ZZ} with $\theta = 2J\Delta t$)

Index mapping ($iter \in \{0, \dots, 2^{n-2}-1\}$):

$$inner = iter \bmod 2^{lo}, \quad mid = \left\lfloor \frac{iter}{2^{lo}} \right\rfloor \bmod 2^{hi-lo-1}, \quad outer = \left\lfloor \frac{iter}{2^{hi-1}} \right\rfloor$$

$$i_{00} = outer \cdot 2^{hi+1} + mid \cdot 2^{lo+1} + inner, \quad i_{01} = i_{00} + 2^{lo}, \quad i_{10} = i_{00} + 2^{hi}, \quad i_{11} = i_{00} + 2^{lo} + 2^{hi}$$

Carving the iterator around both bits lo and hi inserts all four $\{0, 1\}$ combinations, selecting exactly the four coupled amplitudes.



GPU — CUDA Kernel

```

tid = blockIdx × 256 + threadIdx
// one thread per 4-amplitude group

compute inner, mid, outer from tid
compute  $i_{00}, i_{01}, i_{10}, i_{11}$ 

 $\alpha'_{i_{00}} \leftarrow c \alpha_{i_{00}} - is \alpha_{i_{11}}$ 
 $\alpha'_{i_{11}} \leftarrow c \alpha_{i_{11}} - is \alpha_{i_{00}}$ 
 $\alpha'_{i_{01}} \leftarrow c \alpha_{i_{01}} - is \alpha_{i_{10}}$ 
 $\alpha'_{i_{10}} \leftarrow c \alpha_{i_{10}} - is \alpha_{i_{01}}$ 
  
```

CPU — OpenMP

```

#pragma omp parallel for
// one iteration per 4-amplitude group

for  $iter = 0$  to  $2^{n-2} - 1$ :

    compute inner, mid, outer from iter
    compute  $i_{00}, i_{01}, i_{10}, i_{11}$ 

     $\alpha'_{i_{00}} \leftarrow c \alpha_{i_{00}} - is \alpha_{i_{11}}$ 
     $\alpha'_{i_{11}} \leftarrow c \alpha_{i_{11}} - is \alpha_{i_{00}}$ 
     $\alpha'_{i_{01}} \leftarrow c \alpha_{i_{01}} - is \alpha_{i_{10}}$ 
     $\alpha'_{i_{10}} \leftarrow c \alpha_{i_{10}} - is \alpha_{i_{01}}$ 
  
```

MEASURE EXPECTATION

After applying all our gates, we need to extract a single number (The Expectation Value of the 0th qubit, known as the ancilla qubit) from our state vector!

- **Small Recap:**
 - After Step 3, we store copy of our sv
 - **Exp(UNPREP_S -> H[sv]):** Real part of S(k)
 - **Exp(UNPREP_S -> S+ -> H[sv]):** Im(S(k))
 - **Exp(UNPREP_H -> H[sv]):** Real part of H(k)₁
 - **Exp(UNPREP_H -> S+ -> H[sv]):** Im(H(k))₁
- Unprep_H is more nuanced
- **Global Reduction** to obtain the expectation value, giving us the values of our first row of H and S.

What is being computed

The ancilla is qubit 0, so its state lives in the lowest bit of every index i . Even indices have ancilla = 0; odd indices have ancilla = 1.

Per-amplitude contribution ($\text{iter} \in \{0, \dots, 2^n - 1\}$):

$$p_i = \text{Re}(\alpha_i)^2 + \text{Im}(\alpha_i)^2$$

$$\text{if } i \text{ even: } \text{val} += +p_i \quad \text{if } i \text{ odd: } \text{val} += -p_i$$

Result:

$$\langle X \rangle_{\text{anc}} = \sum_{i=0}^{2^n-1} (-1)^{i \bmod 2} p_i = P(\text{anc}=0) - P(\text{anc}=1)$$

Apply Hadamard before measuring $\Rightarrow \text{Re}[S(k)]$. Apply $S^\dagger$ + Hadamard before measuring $\Rightarrow \text{Im}[S(k)]$.



GPU — Shared-Memory Tree Reduction

Each thread strides over the statevector, accumulating its partial sum into a **private register val**.

```
// copy 256 private sums into shared memory
sdata[threadIdx] ← val
```

Tree reduction within block ($\log_2 256 = 8$ steps):

```
for s = 128 → 64 → ... → 1:
    sdata[t] += sdata[t + s]
```

```
// thread 0 writes block total to global
atomicAdd(out, sdata[0])
```

CPU — OpenMP Reduction

```
#pragma omp parallel for
reduction(+:val)
```

```
// one iteration per amplitude
```

```
for iter = 0 to 2n - 1:
```

```
    p ← Re(αi)2 + Im(αi)2
```

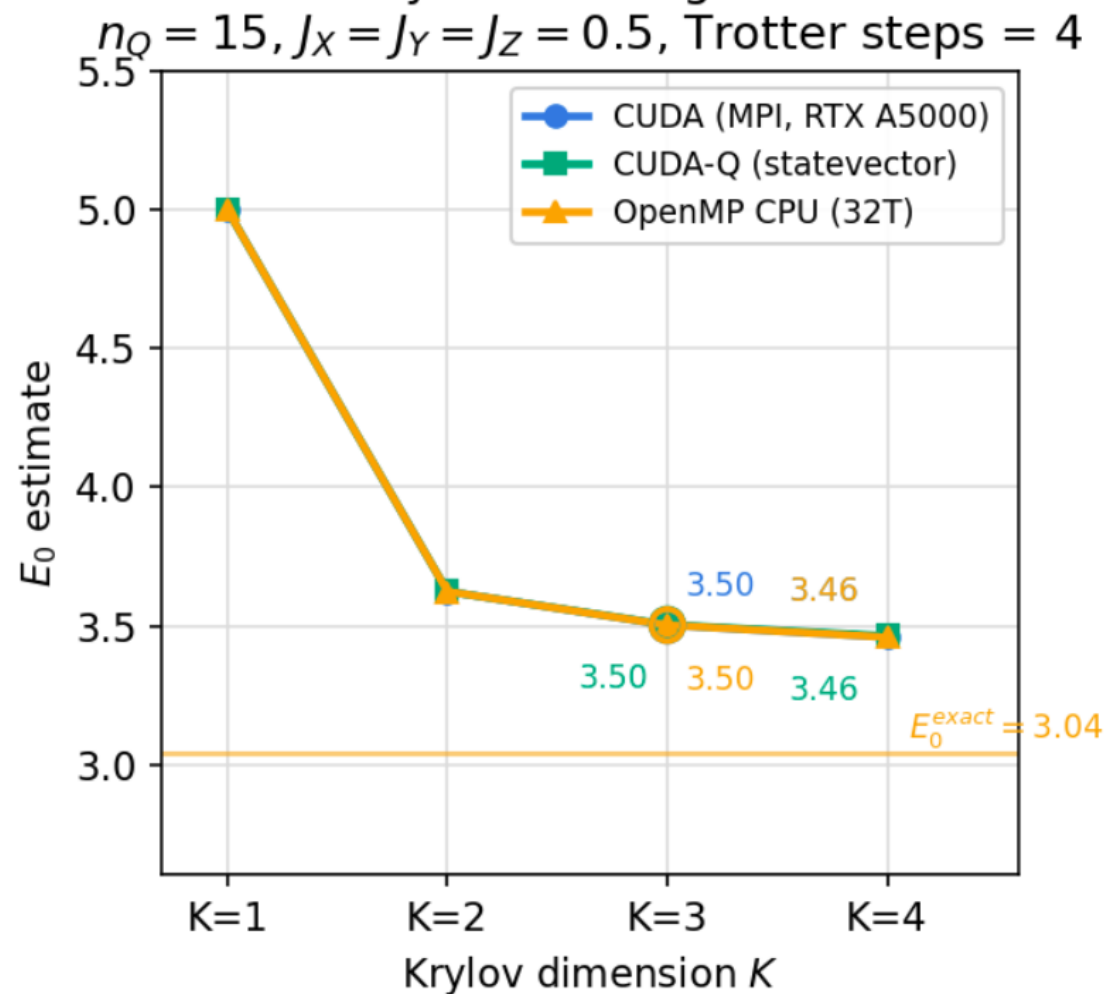
```
    val += (-1)iter mod 2 · p
```

Each thread keeps a private **val**; OpenMP sums all private accumulators at the **parallel-for barrier**.

No shared memory or atomics needed — **reduction(+:val)** handles everything.

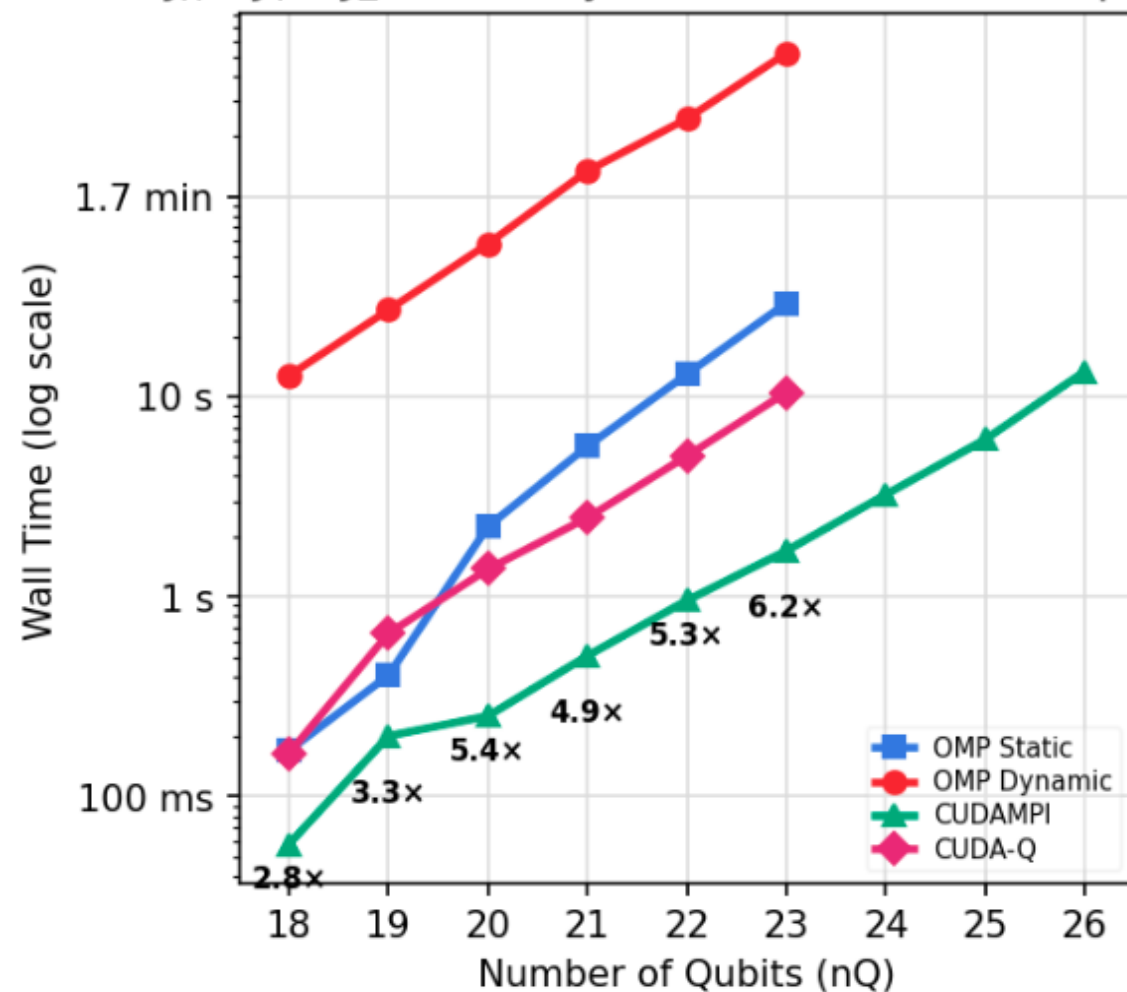
RESULTS

Krylov Convergence



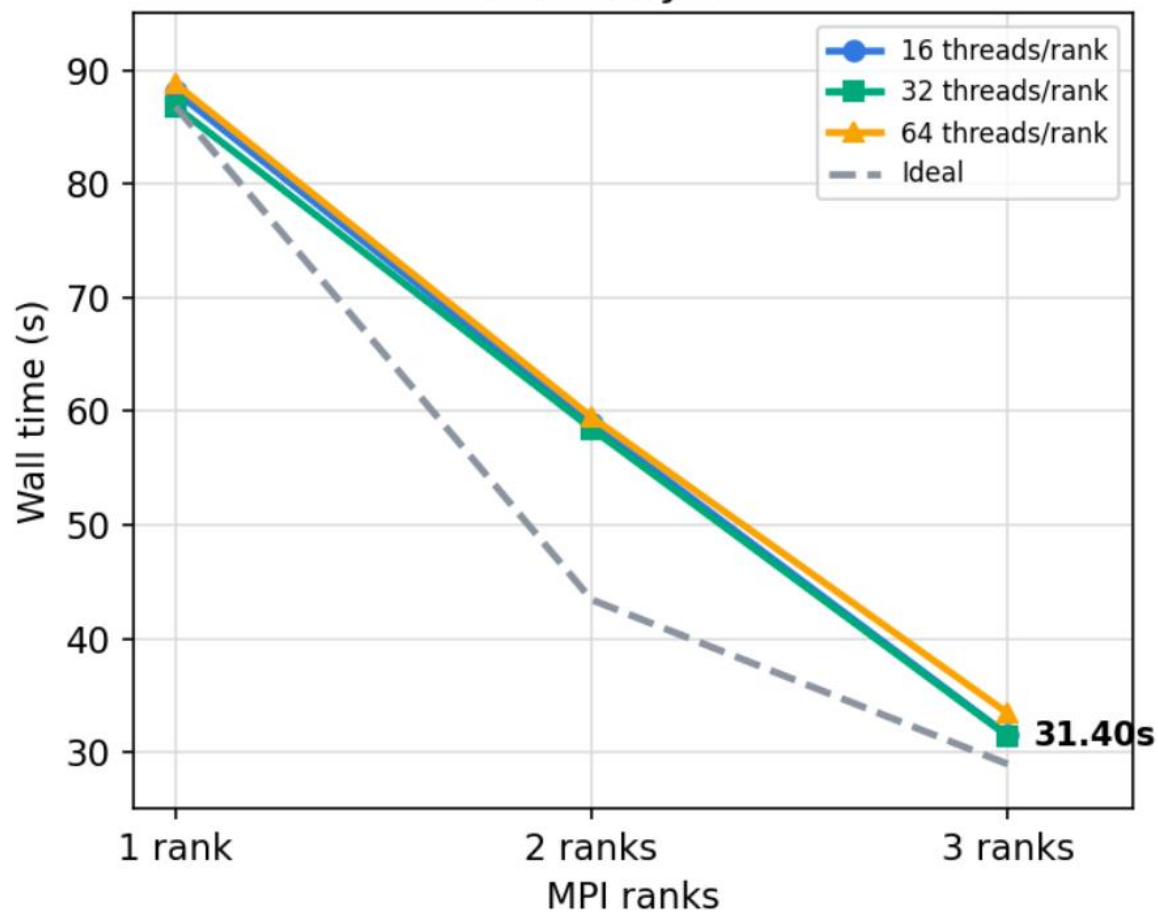
nQ Sweep — Wall Time Comparison

$J_X = J_Y = J_Z = 0.5, \text{ Krylov dim}=4, \text{ Trotter steps}=4$

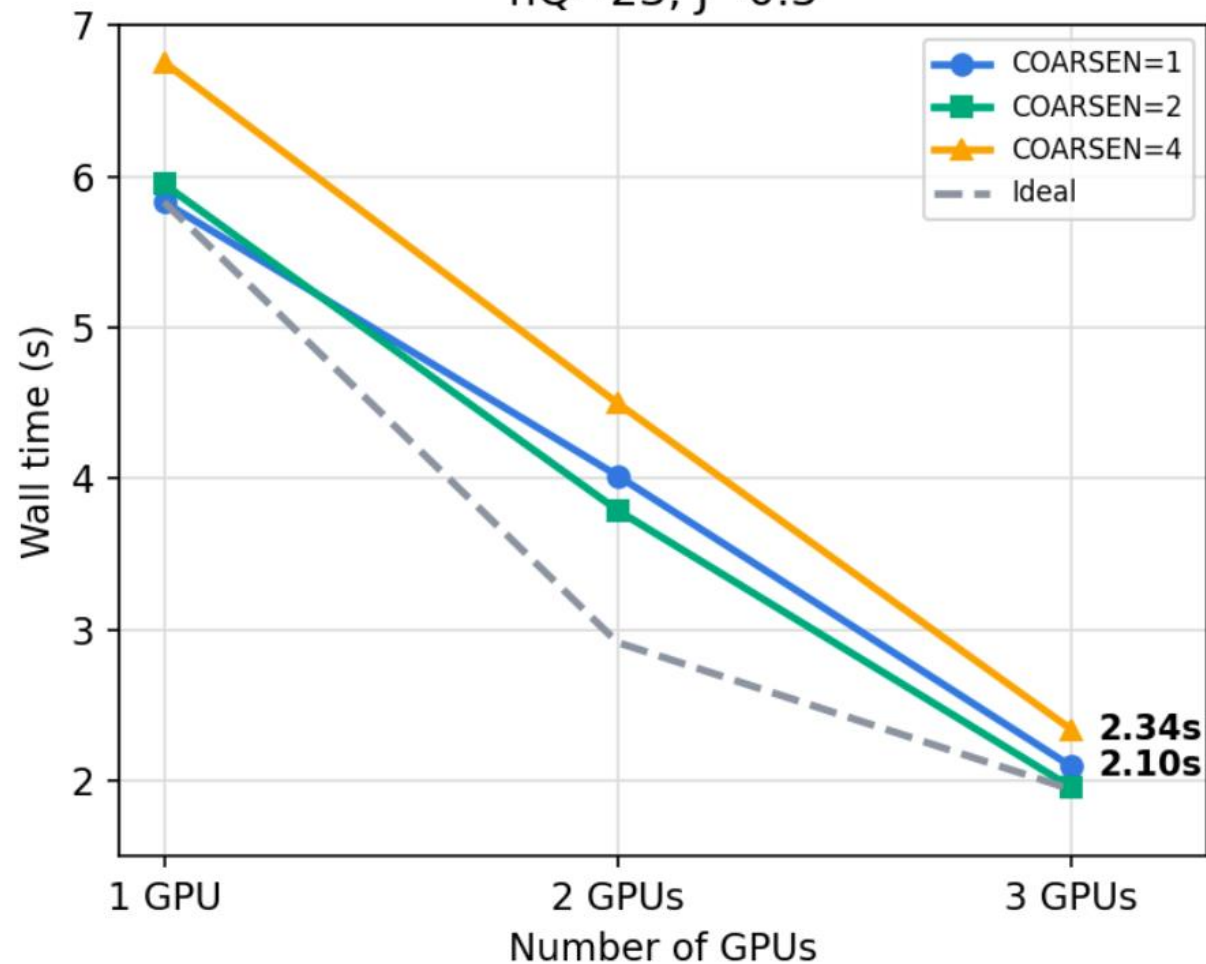


STRONG SCALING RESULTS

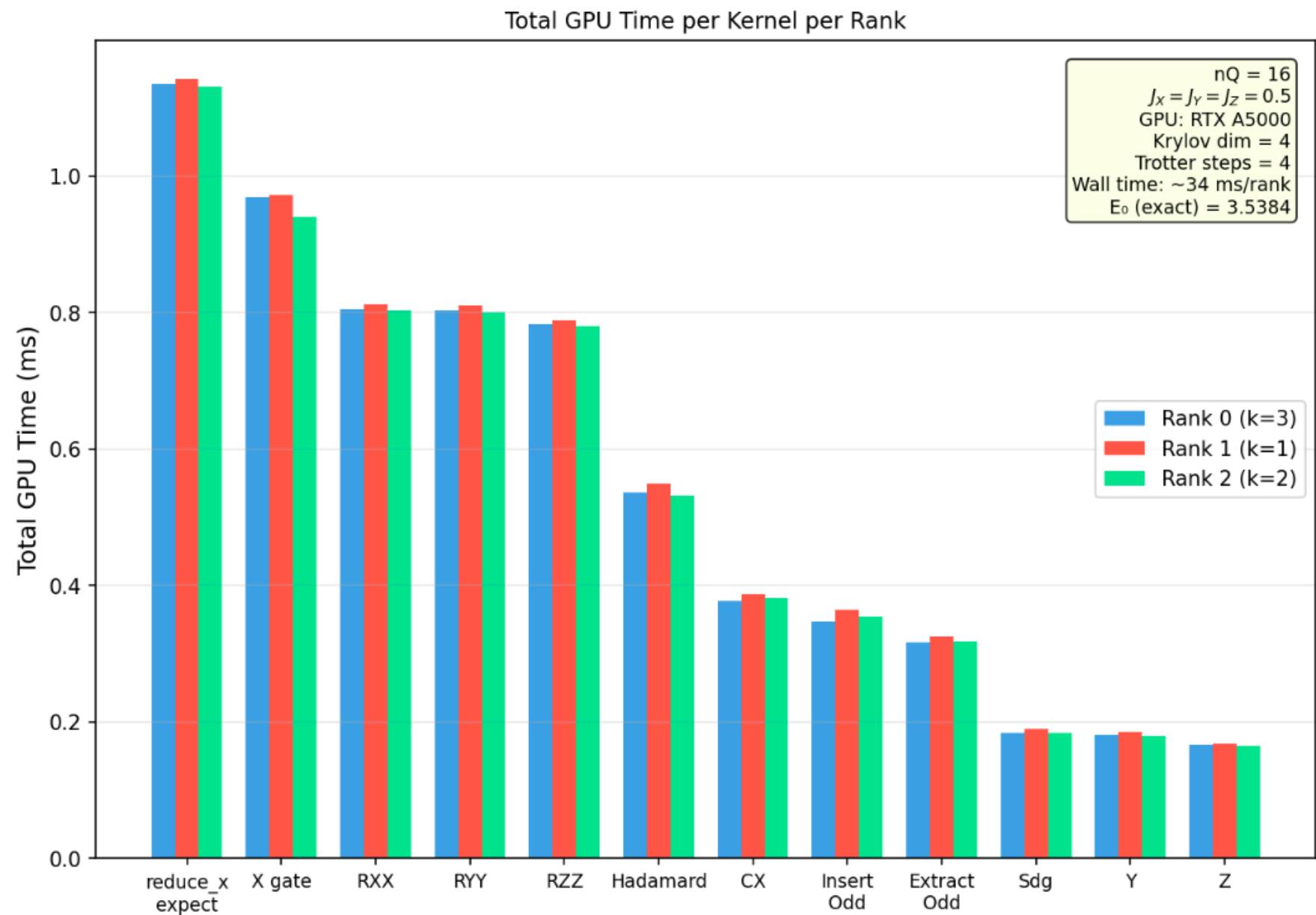
CPU Strong Scaling
 $nQ=23, J=0.5$



GPU Strong Scaling
 $nQ=23, J=0.5$



NSYS PROFILING RESULTS(TOTAL TIME PER KERNEL)



CONCLUSION & EXTENSIONS

CONCLUSION

- Distributed KQD across 3 GPU ranks delivers >6× speedup over the CUDAQ baseline
- The statevector gate kernels (single and two-qubit) scale efficiently
- The measure/reduction operation is the dominant bottleneck at scale
- Distributed KQD is a practical and more efficient path to finding ground state energies of molecules compared to KQD
- The GPU implementation gave far better results than compared to the CPU

Extensions:

- Restructure statevector layout for coalesced memory access — reducing the non-sequential jumps caused by the hi/lo/mid index decomposition for $i_{00}, i_{01}, i_{10}, i_{11}$
- Implement algorithm on **IBM Fez, IBM Marrakesh and IBM Kingston**
 - Currently, 3 QPUs are available to the public
 - **Drawback:** APIs for Inter-QPU parallelism does not exist for consumer use
 - May become available to the public a couple of years from now
- More effective strategies for expectation value computations

THANK YOU! QUESTIONS?

Formula to Find $H(0)$

b counts how many neighbours `excBit(middle qubit)` has. It gets +1 if there is a site to the left (i.e. `excBit` is not the leftmost site), and +1 if there is a site to the right (i.e. `excBit` is not the rightmost site).

```
Complex H0 = Complex(JZ * (bonds_not_touching - bonds_touching), 0.0);  
int bonds_touching    = (excBit > 0 ? 1 : 0) + (excBit < nQ-1 ? 1 : 0);  
int bonds_not_touching = (nQ - 1) - bonds_touching;
```

$$H(0) = JZ \cdot [(N-1-b) - b] = JZ (N-1-2b)$$

SINGLE QUBIT

apply_y_gate : on qubit 3

$$\Rightarrow |i0\rangle + 16 = |i1\rangle$$

$\Rightarrow$ groups: $(0, 16), (1, 17), (2, 18) \dots (15, 31), (32, 48)$
jump

* So, our sv_sys = 1024 size
on q3: offset by 16,

e.g.: $tid = 18$: $Lo = tid \div 2^3 = 2$

$$i0 = hi * 2^{3+1} + 2 = 2^5 + 2 = 34$$

$\Rightarrow i0 = 34, i1 = 50 \Rightarrow (34, 50)$ we multiply $\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$ to these 2 set of vectors

2 QUBIT LOGIC (NEAREST NEIGHBOUR)

*RXX: 100, take qubit (3,4) we want base index = 0

array size = 1024

$\Rightarrow |0\rangle - |0\rangle = \text{SV}[0]$ and so on.
 $|0\rangle - |1\rangle = \text{SV}[1]$

So, for qubit 3 = 0 and qubit 4 = 0, we have:

[0, 1, 2, 3, 4, 5, 6, 7, 32, 33, 34, 35, 36, 37, 38, 39, 64, ...]

$\text{tid} = \text{blockid} \times 256 + \text{threadid}$
tid = 0, 1, 2, 3, ... 255 because we have 2 thread blocks, 256 threads each
at tid = 8:

2 in binary = 1000

7 in binary = 0111

inner = tid / 8 remainder = 0 hi-bit + 1 = 5

outer = tid / 8 = 1

$2 \times 2^5 = 32$

base = (outer * 32) + inner
= 32

at tid = 11: inner = 3, $2^5 = 32$

outer = 1 $\Rightarrow \text{base} = (1 * 32) + 3 = \boxed{35}$

100	=	35
101	=	35 + 8
110	=	35 + 16
111	=	35 + 24

IMPLEMENTATION OF GATES TO OUR STATE VECTOR

Let's take a single-qubit gate first, represented as a unitary 2x2 matrix

We seek to divide our state vector into $2^N/2$ pairs to apply this 2x2 matrix(Algorithm taken from Mikhail et al)

hi, lo: Parameters to map iterator pid to i_0

Each thread is tasked with performing

Examples of Such Gates(Hadamard, X):

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Algorithm 1 Single-Qubit Gate — GPU (CUDA)

Objective: Map each CUDA thread to one amplitude pair $(\alpha_{i_0}, \alpha_{i_1})$ and apply a 2×2 unitary Q across all pairs in parallel. **Variables:** n qubits; qubit index q ; gate $Q \in \mathbb{C}^{2 \times 2}$; $B = 256$ threads/block (constant); coarsening factor C (pairs per thread); $\text{tid} = (\text{blockIdx} \times B + \text{threadIdx}) \times C$; lo, hi bit-field segments; i_0, i_1 statevector indices. **Index formula:** $lo = \text{tid} \bmod 2^q$, $hi = \lfloor \text{tid}/2^q \rfloor$, $i_0 = hi \cdot 2^{q+1} + lo$, $i_1 = i_0 + 2^q$ Inserting 0 or 1 at bit q of tid selects exactly the two amplitudes that differ only at qubit q .

- 1: Launch $\lceil 2^{n-1}/(B \cdot C) \rceil$ blocks of $B = 256$ threads
- 2: **for** $c = 0$ **to** $C - 1$ **do**
- 3: $\text{tid} \leftarrow \text{base} + c$; **if** $\text{tid} \geq 2^{n-1}$ **then return**
- 4: Compute lo, hi, i_0, i_1 via formula above
- 5: $\alpha'_{i_0} \leftarrow q_{11} \cdot \alpha_{i_0} + q_{12} \cdot \alpha_{i_1}$; $\alpha'_{i_1} \leftarrow q_{21} \cdot \alpha_{i_0} + q_{22} \cdot \alpha_{i_1}$
- 6: **end for**

Outcome: 2^{n-1} independent pair updates; no communication between threads.

IMPLEMENTATION OF GATES TO OUR STATE VECTOR

Single Qubit gate for CPU

Algorithm 2 Single-Qubit Gate — CPU (OpenMP)

Objective: Same 2^{n-1} pair updates as Algorithm 1, distributed across CPU threads via a parallel loop instead of a kernel launch. **Variables:** n, q, Q as above; $iter$ loop iterator (0 to $2^{n-1}-1$, plays the role of `tid`); lo, hi bit-field segments; i_0, i_1 statevector indices. **Index formula (identical to GPU):** $lo = iter \bmod 2^q$, $hi = \lfloor iter/2^q \rfloor$, $i_0 = hi \cdot 2^{q+1} + lo$, $i_1 = i_0 + 2^q$

- 1: **parallel for** $iter = 0$ **to** $2^{n-1} - 1$ **do**
- 2: Compute lo, hi, i_0, i_1 via formula above
- 3: $\alpha'_{i_0} \leftarrow q_{11} \cdot \alpha_{i_0} + q_{12} \cdot \alpha_{i_1}$; $\alpha'_{i_1} \leftarrow q_{21} \cdot \alpha_{i_0} + q_{22} \cdot \alpha_{i_1}$
- 4: **end parallel for**

Outcome: Each iteration touches a disjoint pair — no write conflicts. `#pragma omp parallel for` replaces `<<<grid, block>>>`; the index math is unchanged.

IMPLEMENTATION OF GATES TO OUR STATE VECTOR

How the Algorithm looks like using GPUs:

Each CUDA Thread is assigned a logical tid covering the 2^{N-1} amplitude pairs.

Coursening factor C:

Allows each thread to process consecutive pairs for further experimentation

Increase thread compute, reduce number of threads required

Algorithm 3 Two-Qubit Gate — GPU (CUDA) (R_{XX}, R_{YY}, R_{ZZ})

Objective: Map each CUDA thread to one group of four amplitudes $(\alpha_{i_{00}}, \alpha_{i_{01}}, \alpha_{i_{10}}, \alpha_{i_{11}})$ and apply a fused 4×4 rotation across all groups in parallel. **Variables:** n qubits; qubit indices q_1, q_2 ; $lo = \min(q_1, q_2)$, $hi = \max(q_1, q_2)$; angle θ , $c = \cos(\theta/2)$, $s = \sin(\theta/2)$; $B = 256$ threads/block; **tid** global thread index; *inner*, *mid*, *outer* bit-field segments. **Index formula —**

3-segment decomposition:

$$inner = \text{tid} \bmod 2^{lo}, \quad mid = \lfloor \text{tid} / 2^{lo} \rfloor \bmod 2^{hi-lo-1}, \quad outer = \lfloor \text{tid} / 2^{hi-1} \rfloor$$

$$i_{00} = outer \cdot 2^{hi+1} + mid \cdot 2^{lo+1} + inner, \quad i_{01} = i_{00} + 2^{lo}, \quad i_{10} = i_{00} + 2^{hi}, \quad i_{11} = i_{00} + 2^{lo} + 2^{hi}$$

Inserting all four $\{0, 1\}$ combinations at bits lo and hi selects the four coupled amplitudes.

- 1: Launch $\lceil 2^{n-2} / B \rceil$ blocks of $B = 256$ threads
- 2: **tid** $\leftarrow$ blockIdx $\times B$ + threadIdx; **if** **tid** $\geq 2^{n-2}$ **then return**
- 3: Compute $i_{00}, i_{01}, i_{10}, i_{11}$; read $v_{00}, v_{01}, v_{10}, v_{11}$
- 4: $\alpha'_{i_{00}} \leftarrow c v_{00} - i s v_{11}$; $\alpha'_{i_{11}} \leftarrow c v_{11} - i s v_{00}$ $\triangleright 00 \leftrightarrow 11$
- 5: $\alpha'_{i_{01}} \leftarrow c v_{01} - i s v_{10}$; $\alpha'_{i_{10}} \leftarrow c v_{10} - i s v_{01}$ $\triangleright 01 \leftrightarrow 10$

Outcome: 2^{n-2} independent 4-tuple updates. Each Trotter bond (R_{XX}, R_{YY}, R_{ZZ}) is one kernel call; no synchronisation between them.

IMPLEMENTATION OF GATES TO OUR STATE VECTOR

2-Qubit Gate Implementation on CPUs:

Instead of pairs, we obtain groups of 4 for each thread to operate on.

Experimentations with increasing the thread compute with varying C

Example Gates:

$$R_{XX}(\theta) = \exp\left(-i\frac{\theta}{2}X \otimes X\right) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & 0 & 0 & -i\sin\left(\frac{\theta}{2}\right) \\ 0 & \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) & 0 \\ 0 & -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) & 0 \\ -i\sin\left(\frac{\theta}{2}\right) & 0 & 0 & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}$$

$$CX_{q_0, q_1} = I \otimes |0\rangle\langle 0| + X \otimes |1\rangle\langle 1| = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

Algorithm 4 Two-Qubit Gate — CPU (OpenMP) (R_{XX}, R_{YY}, R_{ZZ})

Objective: Same 2^{n-2} four-tuple updates as Algorithm 3, distributed across CPU threads via a parallel loop. **Variables:** $n, q_1, q_2, lo, hi, \theta, c, s$ as above; $iter$ loop iterator (0 to $2^{n-2}-1$, plays the role of `tid`); $inner, mid, outer$ bit-field segments; $i_{00}, i_{01}, i_{10}, i_{11}$ indices. **Index formula (identical to GPU):**

$$inner = iter \bmod 2^{lo}, \quad mid = \lfloor iter/2^{lo} \rfloor \bmod 2^{hi-lo-1}, \quad outer = \lfloor iter/2^{hi-1} \rfloor$$

$i_{00}, i_{01}, i_{10}, i_{11}$ constructed identically to Algorithm 3

- 1: **parallel for** $iter = 0$ **to** $2^{n-2} - 1$ **do**
- 2: Compute $i_{00}, i_{01}, i_{10}, i_{11}$; read $v_{00}, v_{01}, v_{10}, v_{11}$
- 3: $\alpha'_{i_{00}} \leftarrow c v_{00} - i s v_{11}; \quad \alpha'_{i_{11}} \leftarrow c v_{11} - i s v_{00}$
- 4: $\alpha'_{i_{01}} \leftarrow c v_{01} - i s v_{10}; \quad \alpha'_{i_{10}} \leftarrow c v_{10} - i s v_{01}$
- 5: **end parallel for**

Outcome: Each iteration touches a disjoint 4-tuple — no write conflicts. `#pragma omp parallel for` replaces `<<<grid, block>>>`; the index math is unchanged.
